

WireMock - Installation & Setup

A complete guide to prepare the environment, install, configure, validate and perform the first functional use of the tool.

Manual objective

This manual guides you through the complete installation and setup process, from preparing the environment to final validation. Follow the steps in order and use the checks throughout the guide to confirm that each configuration was completed correctly.

Official references

<https://wiremock.org/>

<https://wiremock.org/docs/>

<https://wiremock.org/docs/download-and-installation/>

What WireMock is

WireMock is a tool for simulating HTTP services. It allows a team to replace an unavailable, unstable, expensive or hard-to-control dependency with predictable responses, making tests more repeatable and enabling scenarios that would be difficult to reproduce against a real service.

Stubs and matching

A stub defines which incoming request should match and which response WireMock should return. Matching can consider method, URL, query parameters, headers and body. Keep matchers specific enough to represent the contract, but not so strict that irrelevant details make tests fragile.

Failure scenarios

Mocks are particularly useful for failures: timeouts, 4xx/5xx responses, malformed payloads, slow responses and specific business errors. This helps QA validate resilience and error handling without manipulating a real downstream system.

Standalone or embedded

WireMock can run as a standalone process/container or be embedded in automated tests. Choose based on team architecture, test isolation and CI/CD needs. In either case, version mappings/configuration so scenarios are reproducible.

QA use

QA can use WireMock to unblock integration testing, create deterministic preconditions and reproduce rare dependency behavior. Record which behavior is simulated so results are not mistaken for validation against the real provider.

Attention point

A mock can drift from the real service. Keep stubs aligned with API contracts and periodically validate critical flows against a real integrated environment. Service virtualization complements integration testing; it does not prove the provider itself works.